CSC 370 PaperTrade, Lossless Join Proof

Dryden Bryson

October 16, 2026

Overview

Each of our four relations (`user`, `stock`, `transaction` and `holds`) is already in BCNF (see the *Functional Dependencies and Candidate Keys* document). Losslessness is a property of a *decomposition*, not of a single relation. Our four relations are the result of decomposing the single table we would have without normalization: the **universal relation** U, which holds every attribute of the database in one row per trade.

This document:

1. defines U and its FDs, finds its candidate key, and shows that it violates BCNF, using anomalies from `sql/seed.sql`;
2. decomposes U with the BCNF decomposition algorithm, which ends in exactly our four relations, and proves each step lossless;
3. confirms the whole four-way decomposition is lossless with the chase test;
4. gives a lossy decomposition of `holds` as a counterexample, with spurious tuples taken from the seed data.

1 The Universal Relation

U stores one row per trade. Each row also carries the trader's account details, the stock's details, and the trader's current position in that stock. `transaction` and `holds` both have a `quantity` attribute, so we rename them t_quantity (shares in this trade) and h_quantity (shares currently held).

$$U(\text{transaction_id}, \text{user_id}, \text{name}, \text{email}, \text{ticker}, \text{company_name}, \text{opening_price}, \text{closing_price}, \text{exchange},$$
$$\text{transaction_type}, \text{t_quantity}, \text{price}, \text{time}, \text{h_quantity}, \text{average_price})$$

To keep the expressions readable, we group attributes that always appear together:

$$\begin{aligned} N &= \{\text{name}, \text{email}\} && \text{(account details)} \\ S &= \{\text{company_name}, \text{opening_price}, \text{closing_price}, \text{exchange}\} && \text{(stock details)} \\ T &= \{\text{transaction_type}, \text{t_quantity}, \text{price}, \text{time}\} && \text{(trade details)} \\ P &= \{\text{h_quantity}, \text{average_price}\} && \text{(position details)} \end{aligned}$$

$$U = \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup N \cup S \cup T \cup P$$

Functional Dependencies F

These are the FDs of the four relations, from the *Functional Dependencies and Candidate Keys* document:

- FD_1: $\text{user_id} \to N$
- FD_2: $\text{email} \to \text{user_id}$
- FD_3: $\text{ticker} \to S$
- FD_4: $\text{transaction_id} \to \text{user_id}, \text{ticker}, T$
- FD_5: $\text{user_id}, \text{ticker} \to P$

Candidate Key of U

$$\begin{aligned} \{\text{transaction_id}\}^+ &\supseteq \{\text{transaction_id}\} && \text{(reflexivity)} \\ &\supseteq \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T && \text{(applying } FD_4\text{)} \\ &\supseteq \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T \cup N && \text{(applying } FD_1\text{, transitivity)} \\ &\supseteq \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T \cup N \cup S && \text{(applying } FD_3\text{, transitivity)} \\ &\supseteq \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T \cup N \cup S \cup P && \text{(applying } FD_5\text{, transitivity)} \\ &= U \end{aligned}$$

{transaction_id} is a superkey, and its only proper subset $\emptyset$ has $\emptyset^+ = \emptyset$, so it is a candidate key. No FD has transaction_id on its right-hand side. The only way to derive it is reflexivity, so every superkey must contain transaction_id. {transaction_id} is therefore the **only** candidate key of U.

BCNF Violations

U is in BCNF only if the left-hand side of every non-trivial FD is a superkey. Four of the five FDs fail this:

FD	Closure of left-hand side in U	Superkey?
FD_1	$\{\text{user_id}\}^+ = \{\text{user_id}\} \cup N$	No
FD_2	$\{\text{email}\}^+ = \{\text{user_id}\} \cup N$	No
FD_3	$\{\text{ticker}\}^+ = \{\text{ticker}\} \cup S$	No
FD_4	$\{\text{transaction_id}\}^+ = U$	Yes
FD_5	$\{\text{user_id}, \text{ticker}\}^+ = \{\text{user_id}, \text{ticker}\} \cup N \cup S \cup P$	No

Anomalies in U

Each violation causes anomalies, which the seed data shows directly:

- **Insertion:** every row of U needs a transaction_id. So U cannot store user 9 (Isla MacDonald, who has no trades yet), or the stocks CNR and ATD, which have never been traded.
- **Update:** AAPL appears in 9 trades, so its closing price is stored 9 times. Changing it means updating all 9 rows, and missing one leaves U inconsistent (FD_3).
- **Deletion:** user 7 (Grace Liu) has 4 trades. Deleting those 4 rows also deletes Grace's name and email (FD_1).

2 BCNF Decomposition of U

Lossless join test. A decomposition of R into R_1 and R_2 is lossless if and only if

$$(R_1 \cap R_2) \to R_1 \quad \text{or} \quad (R_1 \cap R_2) \to R_2 \quad \text{is in } F^+$$

That is, the shared attributes form a superkey of at least one side.

Each step of the algorithm picks a violation $X \to Y$ and splits R into $R_1 = X^+$ and $R_2 = (R \setminus X^+) \cup X$. Then $R_1 \cap R_2 = X$, and $X \to X^+ = R_1$ by the definition of closure, so each step passes the test.

Step 1: Remove FD_1

- **Violation:** FD_1: user_id $\to N$, with $\{\text{user_id}\}^+ = \{\text{user_id}\} \cup N$.
- **Split:**

$$R_1 = \{\text{user_id}\} \cup N = \{\text{user_id}, \text{name}, \text{email}\} \implies \texttt{user}$$
$$U_1 = (U \setminus N) \cup \{\text{user_id}\} = \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup S \cup T \cup P$$

- **Lossless:** $R_1 \cap U_1 = \{\text{user_id}\}$, and $\{\text{user_id}\}^+ \supseteq R_1$. So $\{\text{user_id}\}$ is a superkey of R_1 and the split is lossless.
- FD_2 (email $\to$ user_id) now lies entirely inside `user`, where email is a candidate key. So it is no longer a violation.

Step 2: Remove FD_3

The projection of F onto U_1 has the minimal basis FD_3, FD_4 and FD_5 (split into single attributes), as computed in the *Minimal Basis and FD Projection* document. {transaction_id} is still the only key of U_1, so FD_3 still violates BCNF.

- **Violation:** FD_3: ticker $\to S$, with $\{\text{ticker}\}^+ = \{\text{ticker}\} \cup S$.
- **Split:**

$$R_2 = \{\text{ticker}\} \cup S \implies \texttt{stock}$$
$$U_2 = (U_1 \setminus S) \cup \{\text{ticker}\} = \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T \cup P$$

- **Lossless:** $R_2 \cap U_2 = \{\text{ticker}\}$, and $\{\text{ticker}\}^+ \supseteq R_2$. So {ticker} is a superkey of R_2 and the split is lossless.

Step 3: Remove FD_5

The projection of F onto U_2 has the minimal basis FD_4 and FD_5 (see the same document). In U_2, $\{\text{user_id}, \text{ticker}\}^+ \cap U_2 = \{\text{user_id}, \text{ticker}\} \cup P \neq U_2$, because it does not contain transaction_id. So FD_5 still violates BCNF.

- **Violation:** FD_5: user_id, ticker $\to P$.
- **Split:**

$$R_3 = \{\text{user_id}, \text{ticker}\} \cup P = \{\text{user_id}, \text{ticker}, \text{h_quantity}, \text{average_price}\} \implies \texttt{holds}$$
$$R_4 = (U_2 \setminus P) = \{\text{transaction_id}, \text{user_id}, \text{ticker}\} \cup T \implies \texttt{transaction}$$

- **Lossless:** $R_3 \cap R_4 = \{\text{user_id}, \text{ticker}\}$, and $\{\text{user_id}, \text{ticker}\}^+ \supseteq R_3$. So {user_id, ticker} is a superkey of R_3 and the split is lossless.

Result

The algorithm stops, because each of the four resulting relations is in BCNF:

Relation	Attributes	Projected FDs
`user`	user_id, name, email	FD_1, FD_2
`stock`	ticker, company_name, opening_price, closing_price, exchange	FD_3
`holds`	user_id, ticker, h_quantity, average_price	FD_5
`transaction`	transaction_id, user_id, ticker, transaction_type, t_quantity, price, time	FD_4

This is exactly the schema in `sql/ddl.sql`. Every FD of F lies inside one relation, so the decomposition also preserves dependencies.

Each binary step is lossless, so the whole decomposition is lossless. If $R = R_1 \bowtie U_1$ and $U_1 = R_2 \bowtie U_2$, then $R = R_1 \bowtie R_2 \bowtie U_2$, and so on for every step:

$$\pi_{\texttt{user}}(U) \bowtie \pi_{\texttt{stock}}(U) \bowtie \pi_{\texttt{holds}}(U) \bowtie \pi_{\texttt{transaction}}(U) = U$$

3 Chase Test of the Four-Way Decomposition

The chase checks the whole decomposition at once. Build a tableau with one row per relation. Put the distinguished symbol a in each column the relation contains, and a subscripted b_{ij} (row i, column j) everywhere else. Each group N, S, T and P can be one column, because each group lies wholly inside or wholly outside every relation, and every FD's right-hand side contains whole groups.

Initial tableau:

	transaction_id	user_id	N	ticker	S	T	P
`user`	b_{11}	a	a	b_{14}	b_{15}	b_{16}	b_{17}
`stock`	b_{21}	b_{22}	b_{23}	a	a	b_{26}	b_{27}
`holds`	b_{31}	a	b_{33}	a	b_{35}	b_{36}	a
`transaction`	a	a	b_{43}	a	b_{45}	a	b_{47}

Applying the FDs:

1. FD_1 (user_id $\to N$): rows `user`, `holds` and `transaction` agree on user_id $= a$, and `user` has $N = a$. Set $b_{33} := a$ and $b_{43} := a$.

2. FD_3 (ticker $\to S$): rows `stock`, `holds` and `transaction` agree on ticker $= a$, and `stock` has $S = a$. Set $b_{35} := a$ and $b_{45} := a$.

3. FD_5 (user_id, ticker $\to P$): rows `holds` and `transaction` agree on user_id $= a$ and ticker $= a$, and `holds` has $P = a$. Set $b_{47} := a$.

Final tableau:

	transaction_id	user_id	N	ticker	S	T	P
`user`	b_{11}	a	a	b_{14}	b_{15}	b_{16}	b_{17}
`stock`	b_{21}	b_{22}	b_{23}	a	a	b_{26}	b_{27}
`holds`	b_{31}	a	a	a	a	b_{36}	a
`transaction`	a	a	a	a	a	a	a

The `transaction` row is now all distinguished symbols (a), so the decomposition is **lossless**. This agrees with the step-by-step proof.

Joining the Implemented Tables

The proof covers instances of U, where every user, stock and position belongs to at least one trade. The implemented tables can store more than U can, which is how they remove the insertion anomalies. For example, `user` stores Isla MacDonald, who has no trades.

Tuples that have no matching tuple in another relation are **dangling tuples**, and the natural join drops them. In the seed data:

- user 9 and the stocks CNR and ATD join with no trades;
- 16 of the 82 trades are in positions that have since been closed, and the seed data has no `holds` row for a closed position. The join of all four tables therefore returns 66 rows, one for each trade in an open position.

Dangling tuples are data that U could never have held. They do not contradict losslessness: projecting any legal instance of U and joining the projections back still returns exactly that instance.

4 Counterexample: A Lossy Decomposition

To show why the test matters, decompose `holds` so that the shared attributes are not a superkey of either side:

$$H_1 = \{\text{user_id}, \text{h_quantity}\}, \qquad H_2 = \{\text{ticker}, \text{h_quantity}, \text{average_price}\}$$

Then $H_1 \cap H_2 = \{\text{h_quantity}\}$, and $\{\text{h_quantity}\}^+ = \{\text{h_quantity}\}$, so neither $\{\text{h_quantity}\} \to H_1$ nor $\{\text{h_quantity}\} \to H_2$ holds. The test fails.

The seed data has two positions of 33 shares, held by different users in different stocks:

user_id	ticker	h_quantity	average_price
3	AMZN	33	199.6300
8	AAPL	33	226.1091

π_{H_1} contains $(3, 33)$ and $(8, 33)$, and π_{H_2} contains $(\text{AMZN}, 33, 199.6300)$ and $(\text{AAPL}, 33, 226.1091)$. Joining them on h_quantity $= 33$ pairs every user with every stock:

user_id	ticker	h_quantity	average_price	
3	AMZN	33	199.6300	original
3	AAPL	33	226.1091	**spurious**
8	AMZN	33	199.6300	**spurious**
8	AAPL	33	226.1091	original

Over all 22 rows of `holds`, $\pi_{H_1}(\texttt{holds}) \bowtie \pi_{H_2}(\texttt{holds})$ has 24 rows: the 22 originals plus these 2 spurious tuples. The join claims that user 3 holds AAPL and user 8 holds AMZN, which is false. Since the original relation cannot be recovered, the decomposition is **lossy**:

$$\pi_{H_1}(\texttt{holds}) \bowtie \pi_{H_2}(\texttt{holds}) \supsetneq \texttt{holds}$$